

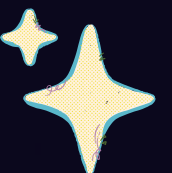
PYTHON PROGRAMMING

INTRODUCTION

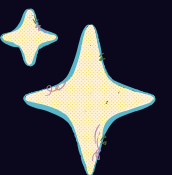
Python is a high-level, interpreted programming language known for its simplicity and readability. Created by Guido van Rossum and first released in 1991, Python emphasizes code readability with its clean syntax, which makes it an excellent choice for both beginners and experienced programmers. It supports multiple programming paradigms, including procedural, object-oriented, and functional programming. Python has a vast standard library and a rich ecosystem of third-party packages, making it suitable for a wide range of applications, from web development to data science, machine learning, automation, and more.



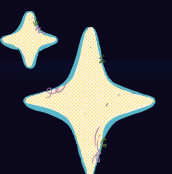
HISTORY



Late 1980s- Python development began by Guido van Rossum as a successor to the ABC language.



1991- Python 0.9.0 was released, introducing core features like functions, exception handling, and basic data structures.



1994 -Python 1.0 was released, adding functional programming tools (map(), filter(), reduce()).



2000- Python 2.0 was released, introducing list comprehensions and garbage collection.



HISTORY CONTINUED.....

- ✧ 2001: Python Software Foundation (PSF) was founded to manage Python's development.
- ✧ 2008: Python 3.0 was released, introducing major changes (e.g., improved Unicode handling, new division operator), breaking backward compatibility.
- ✧ 2020: Python 2.x reached its end of life, focusing the community on Python 3.
- ✧ Present: Python continues to grow in popularity, with applications in web development, data science, AI, and more. Regular updates improve functionality and performance.

FUNCTIONS IN PYTHON

In Python, built-in functions are functions that are always available and do not require any import to use. These functions are provided by Python's standard library and perform common tasks such as data type conversion, mathematical operations, input/output operations, and more.



FUNCTIONS IN PYTHON

Built-in Functions				
abs()	delattr()	hash()	memoryview()	set()
all()	dict()	help()	min()	setattr()
any()	dir()	hex()	next()	slice()
ascii()	divmod()	id()	object()	sorted()
bin()	enumerate()	input()	oct()	staticmethod()
bool()	eval()	int()	open()	str()
breakpoint()	exec()	isinstance()	ord()	sum()
bytearray()	filter()	issubclass()	pow()	super()
bytes()	float()	iter()	print()	tuple()
callable()	format()	len()	property()	type()
chr()	frozenset()	list()	range()	vars()
classmethod()	getattr()	locals()	repr()	zip()
compile()	globals()	map()	reversed()	__import__()
complex()	hasattr()	max()	round()	

FUNCTIONS IN PYTHON

1. Basic Functions

- `print()`: Outputs data to the console.
- `input()`: Reads input from the user.
- `len()`: Returns the length (number of items) of an object.
- `type()`: Returns the type of an object.
- `id()`: Returns the identity of an object.
- `abs()`: Returns the absolute value of a number.
- `min()`, `max()`: Return the smallest/largest item from an iterable.
- `sum()`: Returns the sum of an iterable.

2. Type Conversion

- `int()`, `float()`, `str()`, `bool()`: Convert between data types.
- `list()`, `tuple()`, `set()`, `dict()`: Convert to specific collection types.

3. Iterables and Sequences

- `range()`: Generates a sequence of numbers.
- `enumerate()`: Returns an iterable that yields pairs of index and value.
- `zip()`: Combines two or more iterables, returning tuples of corresponding items.

FUNCTIONS IN PYTHON

4. Functional Programming Tools

- `map()`: Applies a function to all items in an iterable.
- `filter()`: Filters items in an iterable based on a function.
- `sorted()`: Returns a sorted version of an iterable.
- `reversed()`: Returns an iterator that iterates over items in reverse order.

5. Mathematical Functions

- `pow()`: Returns the power of a number.
- `round()`: Rounds a number to a specified number of digits.
- `divmod()`: Returns a tuple of quotient and remainder.

6. Object and Class Handling

- `isinstance()`: Checks if an object is an instance of a class.
- `issubclass()`: Checks if a class is a subclass of another class.
- `hasattr()`, `setattr()`, `getattr()`: Attribute-related functions.
- `dir()`: Lists attributes and methods of an object.

FUNCTIONS IN PYTHON

7. I/O Functions

- `open()`: Opens a file.
- `help()`: Provides documentation for a function/module.

8. Memory and Performance

- `hash()`: Returns the hash value of an object.
- `eval()`, `exec()`: Executes a string as code.

9. Other Utilities

- `any()`, `all()`: Return True if any/all items in an iterable evaluate to True.
- `chr()`, `ord()`: Convert between Unicode characters and their integer codes.
- `bin()`, `hex()`, `oct()`: Convert integers to binary, hexadecimal, or octal strings.

FUNCTIONS IN PYTHON

A user-defined function in Python is a block of reusable code that performs a specific task, defined by the user. It is created using the `def` keyword, followed by a function name, optional parameters, and a colon. The function can return a value using the `return` statement.

Example:

```
python
def greet(name):
    return f"Hello, {name}!"
```

This defines a function `greet()` that takes a `name` argument and returns a greeting string.

Data Science

- Numpy
- Pandas
- SciPy
- Matplotlib
- Sci-Kit Learn

Game Development

- Pygame
- Pyglet
- Panda3D
- Pykyra
- Ursina

Automation

- PyAutoGUI
- Pywinauto
- Selenium
- Requests
- BeautifulSoup

GUI Development

- PyQt5
- Tkinter
- Kivy
- Dear PyGui
- PySide2

Web Development

- Django
- CherryPy
- Flask
- Bottle
- Web2Py

Network Automation

- Netmiko
- NAPALM
- Genie
- NCCClient
- Paramiko

MODULES IN PYTHON

MODULES IN PYTHON

DATA SCIENCE MODULES

NumPy: Fundamental package for numerical computing with arrays.

- Pandas: Data manipulation and analysis.
- Matplotlib: Plotting and visualization.
- Seaborn: Statistical data visualization built on Matplotlib.
- SciPy: Library for scientific and technical computing.
- Scikit-learn: Machine learning library with tools for classification, regression, clustering, etc.
- TensorFlow: Deep learning framework for building neural networks.
- PyTorch: Another deep learning framework that is highly flexible and widely used.
- Statsmodels: Statistical models and hypothesis testing.
- XGBoost: Optimized distributed gradient boosting library.

MODULES IN PYTHON

GAME DEVELOPEMENT MODULES

Pygame: Popular library for building games.

Panda3D: A game engine for Python, focused on 3D rendering and game creation.

- Arcade: A modern Python framework for 2D games.
- Godot-Python: Interface for scripting in Python within the Godot game engine.

MODULES IN PYTHON

AUTOMATION MODULES

» Selenium: Web browser automation.

- BeautifulSoup: Web scraping and parsing HTML/XML documents.
- PyAutoGUI: Cross-platform GUI automation for controlling the mouse and keyboard.
- Requests: For making HTTP requests, useful for interacting with APIs.
- Scrapy: Framework for web scraping and data extraction.
- APScheduler: Scheduling tasks for automation.
- Paramiko: SSH protocol for automating tasks on remote servers.
- PyTest: For writing and running tests to automate testing workflows.

MODULES IN PYTHON

GUI DEVELOPEMENT MODULES

- Tkinter: Standard Python interface to create GUI applications.
 - PyQt: A set of Python bindings for the Qt application framework.
 - Kivy: Open-source Python framework for multi-touch applications.
 - wxPython: A GUI toolkit for the Python language.
 - PyGTK: Python wrapper for GTK+ to develop cross-platform GUIs.
 - PyForms: GUI development framework for building CRUD apps quickly.

MODULES IN PYTHON

WEB DEVELOPEMENT MODULES

- Django: Full-stack web framework for creating robust, scalable web applications.
 - Flask: Micro-framework for web development with minimal setup.
 - FastAPI: Modern web framework for building APIs with high performance.
 - Tornado: Web framework and asynchronous networking library.
 - Bottle: Micro-framework focused on small web applications.
 - Requests: For HTTP/HTTPS requests in web applications.
 - Jinja2: Template engine for Python, often used in web frameworks.
 - SQLAlchemy: ORM (Object-Relational Mapper) for interacting with databases.

MODULES IN PYTHON

NETWORK AUTOMATION MODULES

- Netmiko: Simplifies SSH management of network devices.
- NAPALM: Cross-vendor API to automate network device configurations.
- Paramiko: Python implementation of the SSHv2 protocol.
- Ansible: Automation tool for configuring systems, deploying software, and orchestrating advanced tasks.
- Pexpect: Automates interactions with command-line applications.
- PySNMP: A library for working with SNMP protocol.
- nornir: Network automation framework to manage devices.
- ncclient: Python library for NETCONF clients.

MODULES IN PYTHON

USER DEFINED MODULES



User-defined modules in Python are custom modules created by users to organize and reuse code. A module is simply a Python file containing definitions of functions, classes, variables, and runnable code that can be imported and used in other Python scripts.



THANK YOU